

Architecture Client-Serveur et Programmation Middleware

TP1

Thème	Éléments d'une architecture client-serveur et communication TCP
Public visé	2 ^{ème} Ingénieur
Chargée de cours	Amen Ben Hadj Ali
Chargées de TP	Amen Bidani – Asma Nafti- Arafet Zouari
Année universitaire	2026-2027

Objectifs généraux

Identifier les éléments d'une architecture client-serveur, préciser leurs responsabilités et mettre en œuvre une communication TCP entre des processus clients et un processus serveur.

Le travail commence par l'identification des services et de leur interface. Il se poursuit par la réalisation d'une application permettant à des clients de consulter une bibliothèque gérée par un serveur.

Les données sont conservées en mémoire du côté serveur. Les opérations sont limitées à la consultation et le serveur traite les clients successivement.

Objectifs spécifiques

À l'issue de ce TP, l'étudiant devra être capable de :

- Identifier le client, le serveur, les services et l'interface de service dans une application concrète.
- Distinguer l'interface (le contrat du service), son implémentation et le mécanisme de communication.
- Programmer un échange requête-réponse avec les sockets TCP, entre des programmes exécutés séparément.
- Déployer et tester l'application, d'abord sur un même poste, puis sur des postes distincts du réseau.

Résultat final attendu

Une application dans laquelle les rôles du client, du serveur, du service et de son interface sont identifiables, et dans laquelle plusieurs processus clients peuvent consulter successivement une bibliothèque distante au moyen d'échanges TCP.

Rappels du cours

Les éléments de l'architecture client-serveur

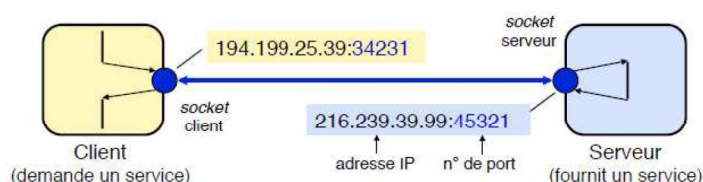
Dans le modèle étudié, un programme client sollicite une fonctionnalité auprès d'un programme serveur. Le serveur reçoit la demande, réalise ou déclenche le traitement nécessaire et transmet une réponse. Les sockets permettent aux deux programmes d'échanger des données sur le réseau.

Pour l'application de ce TP, les quatre éléments sont définis ainsi :

Élément	Rôle dans l'application
Client	Programme qui formule une demande de consultation et présente le résultat à l'utilisateur.
Serveur	Programme qui reçoit les demandes des clients, sollicite les traitements de la bibliothèque et renvoie les réponses.
Service	Fonctionnalité mise à disposition : consulter le catalogue ou rechercher des livres par thème.
Interface de service	Contrat décrivant les opérations accessibles, leurs paramètres, leurs résultats et les erreurs prévues.

Les Sockets en Java

Afin de mettre en œuvre une application client-serveur, il est commode d'utiliser les sockets.



Les sockets constituent une interface de programmation permettant à des processus d'échanger des données. Dans ce TP, nous utilisons des sockets TCP pour établir une communication bidirectionnelle entre deux processus, exécutés sur un même poste ou sur des postes distincts. Chaque extrémité d'une connexion TCP est repérée par une adresse IP et un numéro de port.

Présentation de l'étude de cas

Une bibliothèque universitaire souhaite rendre son catalogue consultable depuis plusieurs postes du réseau. Chaque livre possède un identifiant, un titre et un ensemble de thèmes. Une bibliothèque possède un nom et une collection de livres.

Le catalogue initial est le suivant :

Identifiant	Titre	Thèmes
L1	Programmation Java	JAVA, POO
L2	Réseaux informatiques	RESEAUX, TCP
L3	Systèmes distribués	JAVA, DISTRIBUE
L4	Bases de données	SQL, DONNEES
L5	Architecture logicielle	ARCHITECTURE, POO

Le programme client ne doit pas posséder une copie du catalogue pour réaliser lui-même les recherches. Il transmet la demande au serveur et exploite la réponse reçue. Le serveur conserve le catalogue et propose deux opérations : **consulter** tous les livres et **rechercher** les livres associés à un thème.

Exercice1 — Identification des éléments et préparation des services

1. Analyse de l'architecture

À partir du cas présenté, préciser :

- Le client. Quel programme joue ce rôle ? Quelles informations doit-il transmettre pour demander une recherche ?
- Le serveur. Quelles responsabilités lui appartiennent ? Où doit être conservé le catalogue ?
- Les services. Quelles fonctionnalités seront accessibles aux clients ?
- L'interface. Quelles informations doivent être connues pour utiliser ces fonctionnalités sans connaître leur implémentation ?

Un seul tableau récapitulatif sera demandé dans la synthèse finale. Il précisera les rôles, les classes et les processus effectivement réalisés.

2. Définition de l'interface de service

Le contrat fonctionnel à respecter est le suivant :

Opération	Paramètre	Résultat	Règles
listerLivres	Aucun	Liste de tous les livres	Respecter l'ordre du catalogue.
rechercherParTheme	Un thème non vide	Liste des livres correspondants	Comparer le thème entier, sans tenir compte de la casse et en ignorant les espaces au début et à la fin.

Une recherche sans correspondance retourne une liste vide. Un thème absent ou vide constitue une demande invalide.

Ce tableau constitue **la description de l'interface de service pour cette étape**.

NB : Il n'est pas demandé de créer une interface distante ni d'utiliser un middleware.

3. Implémentation locale des traitements

Le projet de démarrage fournit les classes Livre et Bibliotheque, l'initialisation du catalogue et la méthode `listerLivres()`. Compléter la méthode `rechercherParTheme(String theme)` dans la classe métier `GestionBibliotheque`. Cette classe reçoit la bibliothèque à utiliser et propose les méthodes correspondant aux deux opérations :

```
List<Livre> listerLivres()
List<Livre> rechercherParTheme(String theme)
```

Ces signatures décrivent les deux opérations du service ; elles ne constituent pas, à ce stade, une déclaration d'interface Java.

La classe `GestionBibliotheque` doit contenir uniquement les traitements métier. Elle ne doit ouvrir aucune socket et ne doit pas construire les messages réseau.

Pour un thème `null`, vide ou composé uniquement d'espaces, la méthode de recherche doit signaler un argument invalide par une `IllegalArgumentException`.

4. Validation locale

Créer une classe `TestBibliothequeLocal` et vérifier :

Demande	Résultat attendu
Liste de tous les livres	Les cinq livres.
Recherche de "JAVA"	L1 et L3.
Recherche de " java "	L1 et L3.
Recherche de "HISTOIRE"	Une liste vide.
Recherche avec un thème vide	Une exception signalant l'argument invalide.

NB : À cette étape, les appels sont locaux et les traitements s'exécutent dans un même processus. Aucun serveur n'a encore été mis en œuvre.

Question : que faut-il ajouter pour permettre à un programme exécuté dans un processus distinct, éventuellement sur un autre poste, d'utiliser ces mêmes services ?

Exercice 2 — Première communication entre deux processus avec TCP

Objectif

Établir un échange entre deux programmes indépendants avant de transmettre les demandes de consultation. Cet exercice est guidé : compléter le squelette distribué. Le code obtenu sera repris et adapté dans l'exercice 3.

1. Mécanisme utilisé

En Java, `ServerSocket` permet d'attendre les connexions. L'appel à `accept()` retourne une `Socket` associée au client accepté. Cette socket est utilisée pour échanger les données; elle ne doit pas être confondue avec la socket d'écoute.

La classe `Socket` permet au client d'établir une connexion vers l'adresse et le port du serveur. Les échanges s'effectuent ensuite à travers les flux d'entrée et de sortie de la connexion.

Compléter les deux programmes du squelette fourni : `ServeurBibliothequeTCP`
`ClientBibliothequeTCP`.

Chaque programme possède sa propre méthode `main` et doit être lancé séparément.

2. Convention d'échange

TCP transporte un flux d'octets, sans définir les frontières des messages de l'application.

Il faut donc convenir d'une délimitation. Pour ce TP :

Chaque requête et chaque réponse occupent une ligne de texte, encodée en UTF-8 et terminée par `\n`.

Utiliser des objets de lecture et d'écriture de caractères associés aux flux des sockets. Après l'écriture d'une ligne, vider le tampon d'écriture avec `flush()`. Cet appel n'ajoute pas lui-même le terminateur de ligne.

3. Travail demandé

Côté serveur, attendre un client, accepter sa connexion et lire sa requête. Si la requête est `PING`, envoyer `PONG`. Pour toute autre ligne, envoyer `ERR;COMMANDE_INCONNUE`.

Côté client, établir la connexion, envoyer `PING`, lire la réponse, l'afficher et fermer les ressources.

Le client commence l'échange applicatif. Les messages de diagnostic, tels que « Client connecté », doivent être affichés dans les consoles et non ajoutés aux messages réseau.

Pour cet exercice, le serveur peut s'arrêter après avoir répondu au premier client.

4. Première exécution

Démarrer les deux programmes sur le même poste, dans deux exécutions distinctes, avec:

Adresse du serveur : 127.0.0.1
Port du serveur : 5000

L'adresse 127.0.0.1 est une adresse de boucle locale. Elle sert ici à atteindre le serveur exécuté sur le même poste.

Vérifier la réception de PONG, puis lancer le client alors que le serveur est arrêté. Le client doit signaler clairement l'échec de connexion.

Questions : où commence l'exécution du client ? Où commence celle du serveur ? Qu'est-ce qui distingue cet échange de l'appel local réalisé dans l'exercice 1 ?

Exercice 3 — Accès aux services de la bibliothèque par TCP

Objectif

Rendre les services de consultation de la bibliothèque accessibles à un processus client au moyen d'échanges TCP.

Reprendre le code de l'exercice 2 : l'échange PING/PONG est maintenant remplacé par les commandes de consultation.

1. Traduction de l'interface en messages

L'interface de service définie dans l'exercice 1 est associée au protocole suivant :

Opération du service	Requête TCP
Consulter le catalogue	LISTE
Rechercher par thème	RECHERCHE;theme

Par exemple, pour demander les livres du thème Java, le client transmet :

RECHERCHE;JAVA

Le serveur reconnaît la commande, extrait le thème, puis appelle localement la méthode correspondante de GestionBibliotheque.

Le client TCP n'appelle donc pas directement la méthode Java située dans le processus serveur : il transmet un message que le serveur interprète.

2. Format des réponses

Une réponse réussie contenant des livres suit le format :

OK;nombre;identifiant:titre;identifiant:titre;...

Le champ nombre indique le nombre de livres retournés. Chaque livre est représenté par son identifiant et son titre.

Pour RECHERCHE;JAVA, la réponse attendue est :

OK;2;L1:Programmation Java;L3:Systèmes distribués

Lorsqu'aucun livre ne correspond :

OK; 0

Les erreurs sont représentées ainsi :

ERR;COMMANDE_INCONNUE

ERR;REQUETE_INVALIDE

Les noms des commandes sont écrits en majuscules. Les espaces au début et à la fin des champs sont ignorés. LISTE ne prend aucun argument ; RECHERCHE exige exactement un thème non vide.

Une ligne vide, une commande vide ou un nombre d'arguments incorrect pour une commande connue produit ERR;REQUETE_INVALIDE. Une commande non reconnue produit ERR;COMMANDE_INCONNUE.

Pour ce protocole, les caractères ;, : et les retours à la ligne sont réservés à la structure des messages et ne sont pas autorisés dans les données. Une recherche contenant un caractère réservé dans son thème est invalide.

3. Organisation du serveur

Le serveur doit initialiser une seule bibliothèque et une seule instance de GestionBibliotheque au démarrage.

Pour chaque client, il doit recevoir une requête, l'interpréter, solliciter le service correspondant, construire la réponse et l'envoyer.

Après la réponse, la connexion avec ce client est fermée. Le serveur retourne ensuite à l'attente d'un nouveau client, sans recréer la bibliothèque ni la socket d'écoute.

NB : Par convention dans ce TP, chaque connexion est utilisée pour transmettre une seule requête et une seule réponse, puis elle est fermée. Le serveur traite les clients successivement, sans multithreading.

Séparer le traitement métier, l'interprétation des messages et les échanges réseau. L'interprétation pourra être placée dans une classe auxiliaire ProtocoleBibliotheque.

4. Organisation du client

Le client demande une requête à l'utilisateur, ouvre la connexion, envoie la requête et affiche la réponse reçue.

Il ne doit ni créer le catalogue ni exécuter les méthodes métier de recherche.

L'adresse et le port du serveur doivent être configurables, afin de permettre le passage d'un essai local à une exécution sur plusieurs postes.

5. Traitement des erreurs

Une requête invalide ne doit pas arrêter le serveur.

Si readLine() retourne null, cela signifie que la fin du flux a été atteinte sans lecture d'un nouveau caractère. Cette situation doit être distinguée d'une ligne vide et traitée comme une fin d'échange.

Le client doit différencier une réponse d'erreur métier ou de protocole d'un échec de communication. Une fermeture sans réponse ne doit jamais être affichée comme OK; 0.

Les sockets et les flux doivent être fermés proprement, y compris lorsqu'une erreur survient.

Déploiement sur des postes distincts

1. Exécution sur le réseau

Placer le serveur sur un premier poste et le client sur un second poste.
Configurer le client avec **l'adresse réseau réelle du poste serveur**, et non avec 127.0.0.1. Cette dernière désigne la boucle locale du poste où le client s'exécute.
Vérifier que la socket d'écoute du serveur n'est pas liée uniquement à la boucle locale : elle doit accepter les connexions sur l'adresse réseau utilisée pour l'essai. Java permet de préciser l'adresse locale d'écoute lors de la création ou de la liaison de la `ServerSocket`.
Effectuer cette manipulation sur le réseau autorisé de la salle, avec une règle d'accès au port adaptée et validée par l'enseignant, sans désactiver globalement les protections des postes.

2. Vérification avec plusieurs clients

Exécuter successivement deux clients et vérifier qu'ils accèdent au même catalogue sans redémarrage du serveur.
Pour chaque échange, afficher dans la console du serveur l'adresse distante de la connexion et la requête reçue. La socket connectée fournit notamment l'adresse et le port de son correspondant.

L'objectif est ici de servir plusieurs clients au cours de la vie du serveur, pas de réaliser plusieurs traitements simultanément. Cette seconde étape sera étudiée avec le multithreading (TP2).

3. Tests attendus

Essai	Résultat attendu
LISTE	Les cinq livres du catalogue.
RECHERCHE ; JAVA	L1 et L3.
RECHERCHE ; HISTOIRE	OK ; 0
RECHERCHE ;	ERR ; REQUETE_INVALIDE
SUPPRIMER ; L1	ERR ; COMMANDE_INCONNUE
Requête invalide, puis nouvelle connexion avec LISTE	Le serveur reste opérationnel.
Deux clients utilisés successivement	Les deux demandes sont traitées sans redémarrer le serveur.
Client et serveur sur deux postes distincts	Même résultat fonctionnel que sur un seul poste.
Client exécuté après arrêt du serveur	Échec de connexion signalé clairement.

Un essai réalisé uniquement sur 127.0.0.1 valide l'échange entre processus sur un poste, mais ne constitue pas la validation du déploiement sur deux machines.

Synthèse et travail à remettre

Remettre le projet source, une notice d'exécution et un compte rendu court comprenant les résultats des tests.

Dans un seul tableau récapitulatif, identifier le client, le serveur, le service et l'interface de service dans la version finale. Préciser leurs rôles, les classes et les processus concernés, ainsi que les informations constituant le contrat d'échange.

Répondre aux questions suivantes :

Q1. Service, interface et protocole. Distinguer l'opération de recherche, son contrat et sa représentation dans les messages TCP.

Q2. Communication. Pourquoi le client doit-il envoyer une requête au lieu d'utiliser directement un objet métier créé dans le processus serveur ?

Q3. Évolution. Quels éléments du service pourraient être conservés lors du passage à un middleware d'invocation distante ? Quels éléments du code de communication devraient être remplacés ?